

String Matching

1 Introduction

What is String Matching?

String matching is the problem of finding all occurrences of a **pattern** inside a **text**. It is very important in:

- text editors and search tools,
- DNA sequence analysis,
- search engines and information retrieval,
- pattern detection in large data streams.

Basic Idea

Given a text T and a pattern P , we want to determine all positions where P appears exactly in T .

2 Formal Definition of the Problem

Suppose:

$T[1 : n]$ is the text of length n

$P[1 : m]$ is the pattern of length $m \leq n$

and all characters come from a finite alphabet Σ .

Occurrence of a Pattern

We say that pattern P occurs in text T with **shift** s if

$$0 \leq s \leq n - m$$

and

$$T[s + 1 : s + m] = P[1 : m].$$

Equivalently,

$$T[s + j] = P[j] \quad \text{for } 1 \leq j \leq m.$$

Valid and Invalid Shift

- If the above equality holds, then s is called a **valid shift**.
- Otherwise, s is an **invalid shift**.

Thus, the **string-matching problem** is:

Find all shifts s such that the pattern P occurs in the text T .

3 Example

Let

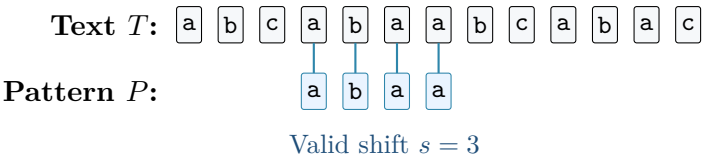
$$T = \text{abcabaabcbac}, \quad P = \text{abaa}.$$

Then P occurs in T only once, at shift

$$s = 3.$$

That means the match starts at position

$$s + 1 = 4.$$



4 Algorithms Mentioned in the Chapter

Most efficient string-matching algorithms perform:

1. a **preprocessing phase** based on the pattern,
2. then a **matching phase** on the text.

Algorithm	Preprocessing Time	Matching Time
Naive	0	$O((n - m + 1)m)$
Rabin–Karp	$\Theta(m)$	$O((n - m + 1)m)$
Finite Automaton	$O(m \Sigma)$	$\Theta(n)$
Knuth–Morris–Pratt (KMP)	$\Theta(m)$	$\Theta(n)$

Key Observation

The naive algorithm does no preprocessing. The other algorithms preprocess the pattern first and then match faster.

5 Notation and Terminology

5.1 Set of All Strings

$$\Sigma^*$$

denotes the set of all finite-length strings formed from the alphabet Σ .

The empty string is denoted by

$$\varepsilon.$$

The length of a string x is denoted by

$$|x|.$$

5.2 Concatenation

If x and y are strings, then their concatenation is written as

$$xy,$$

which means the characters of x followed by the characters of y .

Also,

$$|xy| = |x| + |y|.$$

5.3 Prefix and Suffix

A string w is a **prefix** of a string x if

$$x = wy$$

for some string $y \in \Sigma^*$.

A string w is a **suffix** of a string x if

$$x = yw$$

for some string $y \in \Sigma^*$.

Examples

For the string `abcca`:

- `ab` is a prefix,
- `cca` is a suffix,
- ε is both a prefix and a suffix of every string.

Transitivity

Prefix and suffix relations are transitive:

- if a is a prefix of b and b is a prefix of c , then a is a prefix of c ;
- if a is a suffix of b and b is a suffix of c , then a is a suffix of c .

6 Compact Prefix Notation

For convenience:

$$P_k = P[1 : k]$$

denotes the first k characters of the pattern.

So:

$$P_0 = \varepsilon, \quad P_m = P.$$

Similarly:

$$T_k$$

denotes the first k characters of the text.

Using this notation, the string-matching problem can be restated as:

Compact Form

Find all shifts s such that

P is a suffix of T_{s+m} .

Equivalently,

P matches the last m characters of T_{s+m} .

7 Overlapping-Suffix Lemma

Lemma 1 (Overlapping-Suffix Lemma). *Suppose that x , y , and z are strings such that*

x is a suffix of z and y is a suffix of z .

Then:

- if $|x| \leq |y|$, then x is a suffix of y ;
- if $|x| \geq |y|$, then y is a suffix of x ;
- if $|x| = |y|$, then $x = y$.

Meaning of the Lemma

If two strings are both suffixes of the same larger string, then the shorter one must fit completely at the end of the longer one.

So two suffixes of the same string always **overlap in an ordered way**.

Intuition

Suppose both x and y end at the same final position of z .

Then:

- the shorter suffix lies entirely inside the longer suffix,
- if both have the same length, they must be exactly the same string.

z :

y :

 $|x| \leq |y|$

x :

so x is a suffix of y

8 Cost of Comparing Two Strings

In the discussion of algorithms, comparing two equal-length strings is treated as a primitive operation.

If two strings x and y are compared from left to right and comparison stops at the first mismatch, then the running time is proportional to:

$$\Theta(t + 1),$$

where t is the number of initial matching characters.

Why $t + 1$?

Even if the very first characters do not match, one comparison is still needed. So the cost is written as $\Theta(t + 1)$ instead of $\Theta(t)$.

9 Final Summary

One-box Revision Summary

Text $T[1 : n]$, pattern $P[1 : m]$, $m \leq n$

A shift s is valid if $T[s + 1 : s + m] = P[1 : m]$

Goal: find all valid shifts s , $0 \leq s \leq n - m$

Σ^* = set of all finite strings over Σ , ε = empty string

$P_k = P[1 : k]$, $T_k = T[1 : k]$

Prefix: $x = wy$, Suffix: $x = yw$

If two strings are suffixes of the same string, the shorter is a suffix of the longer

Major algorithms: Naive, Rabin–Karp, Finite Automaton, KMP

Naive String-Matching Algorithm

1 Idea of the Naive Algorithm

Core Idea

The naive string-matching algorithm checks the pattern against the text at **every possible shift**.

At each position, it compares the pattern characters one by one with the corresponding text characters:

$$P[1 : m] \stackrel{?}{=} T[s + 1 : s + m].$$

If all characters match, then that shift is reported as a valid occurrence.

Why is it called “naive”?

It is called naive because it tries all possible alignments directly and does **not use any information** learned from previous mismatches.

2 How the Algorithm Works

Suppose:

$$T[1 : n] = \text{text of length } n, \quad P[1 : m] = \text{pattern of length } m.$$

Then the possible shifts are:

$$s = 0, 1, 2, \dots, n - m.$$

For each shift s , the algorithm checks whether

$$P[1 : m] = T[s + 1 : s + m].$$

If yes, then the pattern occurs at shift s .

Matching Condition

A shift s is valid if

$$T[s + j] = P[j] \quad \text{for every } 1 \leq j \leq m.$$

3 Pseudocode

Naive String Matcher

NAIVE-STRING-MATCHER(T, P)

```
1  n = T.length
2  m = P.length
3  for s = 0 to n - m
4      if P[1 : m] == T[s + 1 : s + m]
5          print "Pattern occurs with shift", s
```

Explanation of Each Step

- Line 1: Find the length of the text.
- Line 2: Find the length of the pattern.
- Line 3: Try every possible shift.
- Line 4: Compare the whole pattern with the current text window.
- Line 5: If they are equal, print the shift.

4 Template View of the Algorithm

The naive algorithm can be imagined as sliding the pattern like a **template** over the text.

Text T :

a	c	a	a	b	c
---	---	---	---	---	---

Pattern P :

a	a	b
---	---	---

First alignment: $s = 0$

At each step, the pattern is shifted one position to the right and compared again.

5 Proper Example 1: Pattern Found Once

Let

$$T = \text{acaabc}, \quad P = \text{aab}.$$

Here:

$$n = 6, \quad m = 3.$$

So the possible shifts are:

$$0, 1, 2, 3.$$

Shift $s = 0$

Compare:

$$T[1 : 3] = \text{aca}, \quad P = \text{aab}.$$

Character comparison:

$$a = a, \quad c \neq a.$$

So this shift is invalid.

Shift $s = 1$

Compare:

$$T[2 : 4] = \text{caa}, \quad P = \text{aab}.$$

First character already mismatches:

$$c \neq a.$$

So this shift is invalid.

Shift $s = 2$

Compare:

$$T[3 : 5] = \text{aab}, \quad P = \text{aab}.$$

All characters match:

$$a = a, \quad a = a, \quad b = b.$$

So the pattern occurs at:

$$s = 2$$

Shift $s = 3$

Compare:

$$T[4 : 6] = \text{abc}, \quad P = \text{aab}.$$

Second character mismatches:

$$a = a, \quad b \neq a.$$

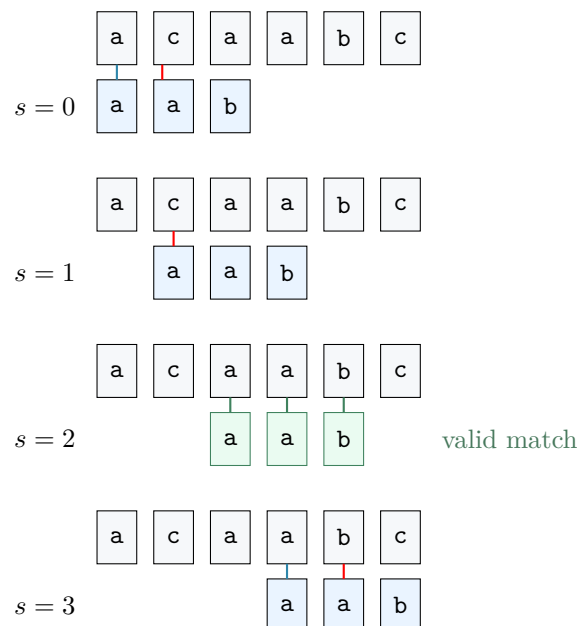
So this shift is invalid.

Conclusion of Example 1

The naive algorithm finds exactly one occurrence of the pattern:

$$\text{Pattern occurs with shift } 2$$

Visual Illustration



6 Proper Example 2: Multiple Comparisons

Let

$$T = \text{aaaaab}, \quad P = \text{aaab}.$$

Then:

$$n = 6, \quad m = 4,$$

and possible shifts are:

$$0, 1, 2.$$

Shift $s = 0$

Compare:

aaaa with aaab

First three characters match, but the fourth mismatches:

$$a = a, \quad a = a, \quad a = a, \quad a \neq b.$$

Shift $s = 1$

Compare:

aaaa with aaab

Again, three characters match and the last one mismatches.

Shift $s = 2$

Compare:

aaab with aaab

All characters match, so this is a valid shift.

Important Lesson

The naive algorithm repeats many comparisons again and again.
Even after learning that several characters matched before, it does not reuse that information.

7 Running Time Analysis

There are exactly

$$n - m + 1$$

possible shifts.

At each shift, in the worst case, the algorithm may compare up to

$$m$$

characters.

Therefore, the worst-case running time is:

$$\mathcal{O}((n - m + 1)m)$$

Worst-Case Time

$$T(n, m) = \Theta((n - m + 1)m)$$

8 Worst-Case Example

Consider:

$$T = a^n \quad (\text{a string of } n \text{ a's})$$

and

$$P = a^m \quad (\text{a string of } m \text{ a's}).$$

For every possible shift:

- all m characters match,
- so each shift requires m comparisons.

Since there are $(n - m + 1)$ shifts, total work is:

$$(n - m + 1)m$$

Hence the algorithm takes:

$$\Theta((n - m + 1)m)$$

If

$$m = \left\lfloor \frac{n}{2} \right\rfloor,$$

then this becomes

$$\Theta(n^2).$$

Why this is bad

So in the worst case, the naive algorithm can become quadratic, which is slow for large texts.

9 Main Limitation of the Naive Algorithm

Key Weakness

The naive algorithm completely ignores useful information from earlier comparisons.

For example, if the pattern is

$$P = \text{aaab}$$

and shift $s = 0$ is valid, then some nearby shifts can be ruled out immediately.

But the naive algorithm still checks them one by one.

So it performs unnecessary repeated work.

Why Better Algorithms Exist

Algorithms like:

- Rabin–Karp,
- Finite Automaton Matcher,
- Knuth–Morris–Pratt (KMP)

try to avoid this repeated comparison.

10 Advantages and Disadvantages

Feature	Discussion
Simplicity	Very easy to understand and implement
Preprocessing	No preprocessing required
Memory usage	Very small
Efficiency	Can be slow in the worst case
Reusability of information	Does not reuse previous matching information
Best use	Small inputs or when simplicity is more important than speed

11 Exam-Oriented Short Note

Short Definition

The naive string-matching algorithm checks the pattern at every possible shift in the text. For each shift, it compares the pattern with the corresponding substring of the text character by character. If all characters match, the shift is reported as a valid occurrence.

Time Complexity

Worst-case time = $\mathcal{O}((n - m + 1)m)$

No preprocessing time

12 Final Summary

One-box Summary

Try every possible shift $s = 0, 1, \dots, n - m$

At each shift, compare $P[1 : m]$ with $T[s + 1 : s + m]$

If all characters match, report the shift

Worst-case running time: $\Theta((n - m + 1)m)$

Main drawback: repeated comparisons are not reused

The Rabin–Karp Algorithm

1 Introduction

Definition

The Rabin–Karp algorithm is a string-matching algorithm that uses **hash values** to compare the pattern with substrings of the text.

Suppose:

$$T[1 : n] = \text{text}, \quad P[1 : m] = \text{pattern}, \quad m \leq n.$$

The basic idea is:

- compute a numeric value for the pattern,
- compute numeric values for each text window of length m ,
- compare these numeric values first,
- verify characters only when the values match.

Why this helps

Direct comparison at every shift may take up to m time per shift. Rabin–Karp avoids this by using a **rolling hash**, so the next window value can be updated quickly.

2 Main Idea of the Algorithm

If the alphabet is digits, for example

$$\Sigma = \{0, 1, 2, \dots, 9\},$$

then each string can be viewed as a number.

Example:

$$31415 \longrightarrow 31415.$$

Let:

$$p = \text{hash value of the pattern } P,$$

and for each shift s ,

$$t_s = \text{hash value of } T[s + 1 : s + m].$$

Then:

$$t_s = p$$

suggests that the current text window may match the pattern.

Important

Hash equality does **not always** guarantee real equality. Sometimes two different strings have the same hash value. This is called a **spurious hit**.

3 Hash Computation

3.1 Pattern Hash

The pattern hash is computed using Horner's rule:

$$p = P[m] + 10(P[m-1] + 10(P[m-2] + \cdots + 10P[1])).$$

This takes:

$$\boxed{\Theta(m)}$$

time.

3.2 First Window Hash

Similarly, the first text window

$$T[1 : m]$$

is converted into

$$t_0.$$

Initial Hashes

$$p = \text{hash of } P[1 : m], \quad t_0 = \text{hash of } T[1 : m].$$

4 Rolling Hash Formula

The most important part of Rabin-Karp is the rolling update.

If we already know t_s , then we compute t_{s+1} without recomputing the whole window.

Rolling Hash Update

For decimal digits,

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1].$$

Meaning

- remove the leftmost digit,
- shift remaining digits to the left,
- add the new rightmost digit.

So the next hash is found in:

$$\boxed{\Theta(1)}$$

time.

5 Why Modular Arithmetic is Used

If m is large, then the numbers become too large.

To fix this, Rabin–Karp computes everything modulo a prime q .

Modular Version

Instead of storing p and t_s , we store:

$$p \bmod q \quad \text{and} \quad t_s \bmod q.$$

For a general radix d , define

$$h \equiv d^{m-1} \pmod{q}.$$

Then the rolling formula becomes:

General Modular Formula

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \pmod{q}.$$

6 Step-by-Step Solved Example 1

Problem

Let

$$P = 31415, \quad m = 5,$$

and suppose the current text window is also

$$31415.$$

Now the next text window is formed by removing the first digit 3 and adding the new digit 2. Find the next hash value.

Solution

Current window:

$$t_s = 31415.$$

Outgoing digit:

$$T[s+1] = 3.$$

Incoming digit:

$$T[s+6] = 2.$$

Using the rolling formula:

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1].$$

Since $m = 5$,

$$10^{m-1} = 10^4 = 10000.$$

So:

$$t_{s+1} = 10(31415 - 10000 \cdot 3) + 2.$$

Now solve step by step:

$$\begin{aligned} t_{s+1} &= 10(31415 - 30000) + 2 \\ &= 10(1415) + 2 \\ &= 14150 + 2 \\ &= \boxed{14152}. \end{aligned}$$

Answer

The next window hash is:

$$\boxed{14152}.$$

7 Step-by-Step Solved Example 2 with Modulo

Problem

Let

$$P = 31415, \quad q = 13.$$

Find the pattern hash modulo 13. Then find the next rolling hash modulo 13 when

$$t_s = 31415, \quad m = 5, \quad d = 10,$$

the outgoing digit is 3, and the incoming digit is 2.

Solution Part A: Pattern Hash Modulo 13

We calculate:

$$31415 \bmod 13.$$

Divide:

$$13 \times 2416 = 31408.$$

So:

$$31415 - 31408 = 7.$$

Hence:

$$\boxed{31415 \equiv 7 \pmod{13}}.$$

So the pattern hash is:

$$\boxed{p = 7}.$$

Solution Part B: Next Window Hash Modulo 13

We use the modular rolling formula:

$$t_{s+1} = (10(t_s - T[s+1]h) + T[s+6]) \pmod{13}.$$

First compute:

$$h = 10^{m-1} \pmod{13} = 10^4 \pmod{13}.$$

Now:

$$10^2 = 100 \equiv 9 \pmod{13},$$

so

$$10^4 \equiv 9^2 = 81 \equiv 3 \pmod{13}.$$

Thus:

$$h = 3.$$

Also:

$$t_s \equiv 7 \pmod{13}.$$

Now substitute:

$$t_{s+1} = (10(7 - 3 \cdot 3) + 2) \pmod{13}.$$

Simplify:

$$\begin{aligned} &= (10(7 - 9) + 2) \pmod{13} \\ &= (10(-2) + 2) \pmod{13} \\ &= (-20 + 2) \pmod{13} \\ &= -18 \pmod{13}. \end{aligned}$$

Since

$$-18 + 26 = 8,$$

we get

$$t_{s+1} \equiv 8 \pmod{13}.$$

Answer

$$p = 7, \quad t_{s+1} = 8 \text{ modulo } 13.$$

8 Spurious Hit Example

Idea

Suppose the pattern hash is

$$p = 7 \pmod{13}.$$

If a text window also has hash

$$t_s = 7 \pmod{13},$$

then we call it a **hit**.

But there are two possibilities:

- it is a **true match**, or
- it is a **spurious hit**.

Example

Pattern:

$$P = 31415.$$

Suppose one text window also gives:

$$t_s = 7 \pmod{13}.$$

Now:

- if the actual substring is 31415, then it is a real match;
- if the actual substring is different but still has remainder 7, then it is a spurious hit.

Conclusion

Hash equality means:

possible match

not

certain match.

So direct character comparison is still necessary after a hit.

9 Algorithm Steps Properly Explained

Rabin-Karp Procedure

```

RABIN-KARP-MATCHER(T, P, d, q)
1  n = T.length
2  m = P.length
3  h = d^(m-1) mod q
4  p = 0
5  t0 = 0
6  for i = 1 to m
7      p = (d*p + P[i]) mod q
8      t0 = (d*t0 + T[i]) mod q
9  for s = 0 to n - m
10     if p == ts
11         if P[1:m] == T[s+1:s+m]
12             print "Pattern occurs with shift", s
13     if s < n - m
14         ts+1 = (d(ts - T[s+1]h) + T[s+m+1]) mod q

```

Explanation of Each Part

- Lines 1–2: read lengths of text and pattern.

- Line 3: compute the value of the highest place $h = d^{m-1} \bmod q$.
- Lines 4–8: compute the pattern hash p and first window hash t_0 .
- Line 9: slide through all valid shifts.
- Line 10: compare hashes.
- Line 11: verify actual strings if hashes match.
- Line 14: compute next window hash in constant time.

10 Full Solved Matching Example

Problem

Use Rabin–Karp to search for

$$P = 26$$

in

$$T = 3141592653589793$$

using

$$d = 10, \quad q = 11.$$

Show the complete matching process.

Given Data

Pattern:

$$P = 26, \quad m = 2$$

Text:

$$T = 3141592653589793, \quad n = 16$$

So the possible shifts are

$$s = 0, 1, 2, \dots, 14.$$

Step 1: Compute h

In Rabin–Karp,

$$h = d^{m-1} \pmod{q}.$$

Since $d = 10$, $m = 2$, and $q = 11$,

$$h = 10^{2-1} \pmod{11} = 10^1 \pmod{11} = 10.$$

So,

$$\boxed{h = 10.}$$

Step 2: Compute the Pattern Hash

The pattern is

$$P = 26.$$

Its hash is

$$p = 26 \pmod{11}.$$

Now,

$$26 = 22 + 4.$$

So,

$$26 \equiv 4 \pmod{11}.$$

Hence,

$$\boxed{p = 4.}$$

Step 3: Compute the First Text Window Hash

The first text window is

$$T[1 : 2] = 31.$$

So,

$$t_0 = 31 \pmod{11}.$$

Since

$$31 = 22 + 9,$$

we get

$$31 \equiv 9 \pmod{11}.$$

Thus,

$$\boxed{t_0 = 9.}$$

Since

$$t_0 \neq p,$$

shift $s = 0$ is rejected.

Step 4: Rolling Hash Computation for All Shifts

The rolling formula is

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \pmod{q}.$$

Since

$$d = 10, \quad h = 10, \quad q = 11,$$

this becomes

$$t_{s+1} = (10(t_s - 10T[s+1]) + T[s+3]) \pmod{11}.$$

We now compute each shift one by one.

Shift $s = 0$

Window:

$$31$$

Hash:

$$t_0 = 9.$$

Compare with pattern hash:

$$9 \neq 4.$$

So shift 0 is rejected.

Compute t_1

Old window: 31, new window: 14

Using the rolling formula:

$$t_1 = (10(t_0 - 3 \cdot 10) + 4) \pmod{11}.$$

Substitute $t_0 = 9$:

$$\begin{aligned} t_1 &= (10(9 - 30) + 4) \pmod{11} \\ &= (10(-21) + 4) \pmod{11} \\ &= -206 \pmod{11}. \end{aligned}$$

Now

$$-206 = -220 + 14,$$

so

$$-206 \equiv 14 \equiv 3 \pmod{11}.$$

Thus,

$$\boxed{t_1 = 3.}$$

Shift $s = 1$

Window:

$$14$$

Hash:

$$t_1 = 3.$$

Compare:

$$3 \neq 4.$$

So shift 1 is rejected.

Compute t_2

Old window: 14, new window: 41

$$t_2 = (10(t_1 - 1 \cdot 10) + 1) \pmod{11}.$$

Substitute $t_1 = 3$:

$$\begin{aligned} t_2 &= (10(3 - 10) + 1) \pmod{11} \\ &= (10(-7) + 1) \pmod{11} \\ &= -69 \pmod{11}. \end{aligned}$$

Since

$$-69 = -77 + 8,$$

we get

$$\boxed{t_2 = 8.}$$

Shift $s = 2$

Window:

$$41$$

Hash:

$$t_2 = 8.$$

Compare:

$$8 \neq 4.$$

So shift 2 is rejected.

Compute t_3

Old window: 41, new window: 15

$$t_3 = (10(t_2 - 4 \cdot 10) + 5) \pmod{11}.$$

Substitute $t_2 = 8$:

$$\begin{aligned} t_3 &= (10(8 - 40) + 5) \pmod{11} \\ &= (10(-32) + 5) \pmod{11} \\ &= -315 \pmod{11}. \end{aligned}$$

Since

$$-315 = -319 + 4,$$

we get

$$\boxed{t_3 = 4.}$$

Shift $s = 3$

Window:

$$15$$

Hash:

$$t_3 = 4.$$

Now

$$t_3 = p.$$

So this is a **hit**. Now we verify the actual strings:

$$15 \neq 26.$$

So shift 3 is a **spurious hit**.

Compute t_4

Old window: 15, new window: 59

$$t_4 = (10(t_3 - 1 \cdot 10) + 9) \pmod{11}.$$

Substitute $t_3 = 4$:

$$\begin{aligned} t_4 &= (10(4 - 10) + 9) \pmod{11} \\ &= (10(-6) + 9) \pmod{11} \\ &= -51 \pmod{11}. \end{aligned}$$

Since

$$-51 = -55 + 4,$$

we get

$$\boxed{t_4 = 4.}$$

Shift $s = 4$

Window:

$$59$$

Hash:

$$t_4 = 4.$$

Again,

$$t_4 = p.$$

So this is another hit.

Verify:

$$59 \neq 26.$$

Hence shift 4 is also a **spurious hit**.

Compute t_5

Old window: 59, new window: 92

$$t_5 = (10(t_4 - 5 \cdot 10) + 2) \pmod{11}.$$

Substitute $t_4 = 4$:

$$\begin{aligned} t_5 &= (10(4 - 50) + 2) \pmod{11} \\ &= (10(-46) + 2) \pmod{11} \\ &= -458 \pmod{11}. \end{aligned}$$

Since

$$-458 = -462 + 4,$$

we get

$$\boxed{t_5 = 4.}$$

Shift $s = 5$

Window:

$$92$$

Hash:

$$t_5 = 4.$$

So this is a hit.

Verify:

$$92 \neq 26.$$

Thus shift 5 is also a **spurious hit**.

Compute t_6

Old window: 92, new window: 26

$$t_6 = (10(t_5 - 9 \cdot 10) + 6) \pmod{11}.$$

Substitute $t_5 = 4$:

$$\begin{aligned} t_6 &= (10(4 - 90) + 6) \pmod{11} \\ &= (10(-86) + 6) \pmod{11} \\ &= -854 \pmod{11}. \end{aligned}$$

Since

$$-854 = -858 + 4,$$

we get

$$\boxed{t_6 = 4.}$$

Shift $s = 6$

Window:

26

Hash:

$t_6 = 4$.

So this is a hit.

Now verify actual strings:

$26 = 26$.

Therefore, shift 6 is a **valid match**.

First Real Match

The pattern occurs at

$s = 6$.

Continue Remaining Shifts

We continue the same process for the rest of the windows.

Shift s	Window	Hash mod 11	Result
0	31	9	reject
1	14	3	reject
2	41	8	reject
3	15	4	hit, but spurious
4	59	4	hit, but spurious
5	92	4	hit, but spurious
6	26	4	valid match
7	65	10	reject
8	53	9	reject
9	35	2	reject
10	58	3	reject
11	89	1	reject
12	97	9	reject
13	79	2	reject
14	93	5	reject

Verification of Remaining Hashes

For completeness, the remaining window remainders are:

$$65 \equiv 10 \pmod{11}, \quad 53 \equiv 9 \pmod{11}, \quad 35 \equiv 2 \pmod{11},$$

$$58 \equiv 3 \pmod{11}, \quad 89 \equiv 1 \pmod{11}, \quad 97 \equiv 9 \pmod{11},$$

$$79 \equiv 2 \pmod{11}, \quad 93 \equiv 5 \pmod{11}.$$

None of these equals the pattern hash 4, so no more candidate matches occur.

Final Conclusion

The Rabin–Karp algorithm finds:

- **spurious hits** at shifts

$$3, 4, 5,$$

because their hash values equal the pattern hash but the actual strings do not match;

- one **real match** at shift

$$\boxed{6}.$$

Final Answer

For

$$P = 26, \quad T = 3141592653589793, \quad d = 10, \quad q = 11,$$

the Rabin–Karp algorithm reports:

$$\boxed{\text{Pattern occurs with shift 6.}}$$

It also encounters $\boxed{3}$ spurious hits, at shifts

$$\boxed{3, 4, 5}.$$

11 Running Time Analysis

11.1 Preprocessing Time

We compute:

- h ,
- pattern hash p ,
- first window hash t_0 .

So preprocessing time is:

$$\boxed{\Theta(m)}.$$

11.2 Matching Time

There are

$$n - m + 1$$

shifts.

At each shift:

- hash comparison takes constant time,
- rolling update takes constant time,
- but exact verification after a hit may take m time.

So worst-case matching time is:

$$\Theta((n - m + 1)m).$$

11.3 Worst Case Example

If

$$P = a^m \quad \text{and} \quad T = a^n,$$

then every shift becomes a hit and must be fully verified.

Thus total time is:

$$\Theta((n - m + 1)m).$$

11.4 Expected Case

If spurious hits are rare and actual matches are few, then the algorithm behaves much better in practice, often near:

$$\mathcal{O}(n + m).$$

12 Advantages and Disadvantages

Point	Description
Main strength	Fast filtering using hash values
Update cost	Next window hash in $\Theta(1)$ time
Preprocessing	$\Theta(m)$
Worst case	$\Theta((n - m + 1)m)$
Main weakness	Spurious hits due to collisions
Practical benefit	Often much faster than naive matching in average cases

13 Final Summary

One-Box Summary

Rabin–Karp compares hash values instead of comparing full strings first

Pattern hash p and window hash t_s are computed

Rolling hash update: $t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$

If $t_s = p$, verify characters directly

Spurious hit: same hash but different strings

Preprocessing time: $\Theta(m)$

Worst-case matching time: $\Theta((n-m+1)m)$

Expected practical performance is often much better

String Matching with Finite Automata

1 Core Idea

What the Automaton Stores

For pattern $P[1 : m]$, state q means:

the first q characters of P match the current text suffix.

State m means the whole pattern has matched.

The automaton scans the text once. It never moves backward in the text. All fallback decisions are already stored in the transition table.

States

$$Q = \{0, 1, 2, \dots, m\}, \quad q_0 = 0, \quad A = \{m\}.$$

2 Suffix Function

Definition

Let $P_k = P[1 : k]$. Define

$$\sigma(x) = \max\{k \mid P_k \text{ is a suffix of } x\}.$$

So $\sigma(x)$ is the length of the longest prefix of P that appears at the end of x .

Example with $P = \mathbf{ab}$	Longest prefix of P at the end	$\sigma(x)$
$x = \varepsilon$	none	0
$x = \mathbf{ccaca}$	a	1
$x = \mathbf{ccab}$	ab	2

Match Test

$$\sigma(T_i) = m \iff P \text{ ends at position } i.$$

3 Transition Function

Transition Rule

If the current state is q and the next character is a , then

$$\delta(q, a) = \sigma(P_q a).$$

Meaning: append the new character to the currently matched prefix, then keep the longest prefix of P that is still a suffix.

Case	What happens
Next character matches	If $a = P[q + 1]$, then $\delta(q, a) = q + 1$.
Mismatch	Fall back to the largest smaller prefix of P that is still a suffix.

4 Key Lemmas

Lemma 1 (One Character Can Increase the Match by At Most One). *For any string x and character a ,*

$$\sigma(xa) \leq \sigma(x) + 1.$$

Proof. If $\sigma(xa) = r$, then P_r is a suffix of xa . Remove the last character from both strings. Then P_{r-1} is a suffix of x . Therefore

$$r - 1 \leq \sigma(x),$$

so $r \leq \sigma(x) + 1$. □

Lemma 2 (State q Is Enough Information). *If $q = \sigma(x)$, then for any character a ,*

$$\sigma(xa) = \sigma(P_q a).$$

Proof. State q says that the useful suffix of x is exactly P_q . After reading a , any new match must be a suffix of $P_q a$. Therefore the next state can be computed from $P_q a$, not from the full old text x . □

5 Matcher Algorithm

Finite-Automaton Matcher

```
FINITE-AUTOMATON-MATCHER(T, delta, m)
1  q = 0
2  for i = 1 to n
3      q = delta(q, T[i])
4      if q == m
5          print "Pattern occurs with shift", i - m
```

Why the Shift is $i - m$

When state m is reached at text position i , the pattern occupies positions

$$i - m + 1, \dots, i.$$

So the 0-based shift is $i - m$.

6 Correctness

Theorem 1 (Main Invariant). *After reading T_i , the current automaton state is*

$$\phi(T_i) = \sigma(T_i).$$

Proof. Use induction on i . Initially, $T_0 = \varepsilon$, so the state is $0 = \sigma(\varepsilon)$.

Assume after reading T_i , the state is $q = \sigma(T_i)$. Let the next character be $a = T[i + 1]$. The automaton moves to

$$\delta(q, a) = \sigma(P_q a).$$

By Lemma 2,

$$\sigma(P_q a) = \sigma(T_i a) = \sigma(T_{i+1}).$$

So the invariant remains true. □

Correctness Result

The matcher reports exactly when $q = m$. By the invariant, this is exactly when $\sigma(T_i) = m$, meaning P is a suffix of T_i . Therefore every reported shift is correct.

7 Building the Transition Table

Direct Construction

```

COMPUTE-TRANSITION-FUNCTION(P, Sigma)
1  m = P.length
2  for q = 0 to m
3      for each character a in Sigma
4          k = min(m+1, q+2)
5          repeat
6              k = k - 1
7              until P[1:k] is a suffix of P[1:q]a
8          delta(q,a) = k
9  return delta

```

Manual Rule

To compute $\delta(q, a)$: write $P_q a$, then choose the longest prefix of P that is a suffix of that string.

8 Worked Example

Let

$$P = ab, \quad \Sigma = \{a, b, c\}.$$

The states are 0, 1, 2.

State q	read a	read b	read c
0	1	0	0
1	1	2	0
2	1	0	0

Why $\delta(2, \mathbf{a}) = 1$: after **ab** then **a**, we have **aba**. The longest prefix of **ab** that is a suffix of **aba** is **a**.
Scan $T = \text{ccab}$:

i	$T[i]$	State q	Meaning
1	c	0	no useful match
2	c	0	no useful match
3	a	1	matched a
4	b	2	matched ab

At $i = 4$, state $2 = m$. Therefore the match has shift

$$i - m = 4 - 2 = 2.$$

9 Complexity

Part	Time
Build transition table directly	$O(m^3 \Sigma)$
Build transition table with improved method	$O(m \Sigma)$
Scan text after table is built	$\Theta(n)$

Important

The matching phase is always linear. The expensive part is preprocessing the transition table.

10 Final Summary

One-Box Summary

State q means $P[1 : q]$ matches the current text suffix.
 $\sigma(x) = \max\{k \mid P[1 : k] \text{ is a suffix of } x\}$.
 $\delta(q, a) = \sigma(P_q a)$.
 $\phi(T_i) = \sigma(T_i)$ is the main invariant.
 $q = m \Rightarrow$ pattern found with shift $i - m$.
 Matching time after preprocessing is $\Theta(n)$.

The Knuth–Morris–Pratt (KMP) Algorithm

1 Introduction

Main Idea

KMP searches for a pattern $P[1 : m]$ in a text $T[1 : n]$ without rechecking characters unnecessarily. It uses the pattern's own structure to decide where to continue after a mismatch.

Running Time

Preprocessing $\Theta(m)$,	matching $\Theta(n)$,	total $\Theta(n + m)$.
-----------------------------	------------------------	-------------------------

The key precomputed array is the **prefix function** $\pi[1 : m]$. It tells KMP how much of the current match can still be reused.

2 The Prefix Function

Prefix Function

For each $q = 1, 2, \dots, m$,

$$\pi[q] = \max\{k \mid k < q \text{ and } P_k \text{ is a suffix of } P_q\}.$$

Interpretation

If a mismatch occurs after matching q characters, then KMP falls back to

$$q \leftarrow \pi[q].$$

So $\pi[q]$ is the reusable matched length.

3 Worked Example of the Prefix Function

Let

$$P = \text{ababaca}.$$

We compute $\pi[q]$ for each position.

q	$P[q]$	Prefix $P[1 : q]$	$\pi[q]$
1	a	a	0
2	b	ab	0
3	a	aba	1
4	b	abab	2
5	a	ababa	3
6	c	ababac	0
7	a	ababaca	1

Thus:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

4 How KMP Uses the Prefix Function

If q characters of P have matched and the next character fails, KMP does not restart. It falls back to

$$q \leftarrow \pi[q],$$

which keeps the longest prefix that can still be reused.

5 The KMP Matcher Algorithm

KMP Matcher

```

KMP-MATCHER(T, P)
1  n = T.length
2  m = P.length
3  pi = COMPUTE-PREFIX-FUNCTION(P)
4  q = 0
5  for i = 1 to n
6      while q > 0 and P[q+1] != T[i]
7          q = pi[q]
8      if P[q+1] == T[i]
9          q = q + 1
10     if q == m
11         print "Pattern occurs with shift", i - m
12         q = pi[q]
```

Reading the Algorithm

i scans the text and q stores how many pattern characters are currently matched. On a mismatch, lines 6–7 shrink q using $\pi[q]$. On a full match, line 12 again uses $\pi[q]$, so overlapping matches are not missed.

6 Worked Matching Example

Let

$$P = \text{ababaca}, \quad T = \text{abababacaba}.$$

We already know:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

Now track the matching process.

Step-by-step

i	$T[i]$	q after processing	Comment
1	a	1	matched $P[1]$
2	b	2	matched $P[2]$
3	a	3	matched $P[3]$
4	b	4	matched $P[4]$
5	a	5	matched $P[5]$
6	b	4	mismatch with $P[6] = c$, fallback to $\pi[5] = 3$, then match b
7	a	5	matched again
8	c	6	matched $P[6]$
9	a	7	full pattern matched
10	b	2	fallback after full match
11	a	3	continue scanning

At $i = 9$, we have:

$$q = 7 = m.$$

So the pattern occurs with shift:

$$9 - 7 = 2.$$

Result

The pattern **ababaca** occurs in the text **abababacaba** at shift

2.

7 Computing the Prefix Function

The prefix function itself is computed efficiently by another linear-time procedure.

Compute Prefix Function

```
COMPUTE-PREFIX-FUNCTION(P)
1  m = P.length
2  let pi[1:m] be a new array
3  pi[1] = 0
4  k = 0
5  for q = 2 to m
6      while k > 0 and P[k+1] != P[q]
7          k = pi[k]
8      if P[k+1] == P[q]
9          k = k + 1
10     pi[q] = k
11 return pi
```

8 Meaning of the Prefix-Function Algorithm

Here:

- q is the current position whose prefix value we want,
- k stores the current best candidate for $\pi[q - 1]$.

The algorithm extends the previous best border if possible. If not, it repeatedly falls back using the already computed prefix values.

Key Insight

Just as KMP avoids re-comparing text characters, the prefix-function algorithm avoids re-comparing pattern characters unnecessarily.

9 Worked Example: Computing π

Let

$$P = \text{ababaca}.$$

We compute π step by step.

Initialization

$$\pi[1] = 0, \quad k = 0.$$

For $q = 2$

Compare:

$$P[k + 1] = P[1] = a, \quad P[2] = b.$$

Mismatch, so:

$$\pi[2] = 0.$$

For $q = 3$

Compare:

$$P[1] = a, \quad P[3] = a.$$

Match, so:

$$k = 1, \quad \pi[3] = 1.$$

For $q = 4$

Compare:

$$P[2] = b, \quad P[4] = b.$$

Match, so:

$$k = 2, \quad \pi[4] = 2.$$

For $q = 5$

Compare:

$$P[3] = a, \quad P[5] = a.$$

Match, so:

$$k = 3, \quad \pi[5] = 3.$$

For $q = 6$

Compare:

$$P[4] = b, \quad P[6] = c.$$

Mismatch, so fallback:

$$k = \pi[3] = 1.$$

Now compare:

$$P[2] = b, \quad P[6] = c.$$

Mismatch again, fallback:

$$k = \pi[1] = 0.$$

Now compare:

$$P[1] = a, \quad P[6] = c.$$

Still mismatch, so:

$$\pi[6] = 0.$$

For $q = 7$

Compare:

$$P[1] = a, \quad P[7] = a.$$

Match, so:

$$k = 1, \quad \pi[7] = 1.$$

Thus:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

10 Running-Time Analysis of Compute-Prefix-Function

At first glance, the while loop seems expensive. But overall, the algorithm is linear.

Why?

- k increases only when a match occurs.
- k decreases only through:

$$k = \pi[k].$$

- k can never become negative.

So across the entire algorithm:

- total increases of k are at most $m - 1$,
- total decreases of k are also at most $m - 1$.

Therefore the total running time is:

$$\Theta(m).$$

Conclusion

The prefix function is computed in linear time.

11 Running-Time Analysis of KMP Matcher

The matcher also runs in linear time.

Why?

As the text is scanned:

- i increases from 1 to n ,
- q increases when characters match,
- q decreases only through prefix-function fallback.

Just as before, total increases and decreases of q are both bounded by n .

So total work is:

$$\Theta(n).$$

Thus the overall KMP complexity is:

$$\Theta(n + m).$$

Why is the matching time linear?

The variable q measures how many pattern characters are currently matched. During the whole scan:

- q can increase at most n times, once per text character,
- every fallback sets $q = \pi[q]$, so q strictly decreases,
- q never goes below 0.

Therefore the total number of fallbacks is also at most n . So the matching phase takes:

$$\boxed{\Theta(n)}.$$

Including prefix computation, the total running time is:

$$\boxed{\Theta(n + m)}.$$

12 Prefix-Function Iteration Lemma

A key theoretical result is that repeated application of π lists all proper prefix-suffix lengths.

Lemma 1 (Prefix-Function Iteration Lemma). *For a pattern P , the values*

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

list all lengths $k < q$ such that P_k is a suffix of P_q .

Meaning

If you start at q and repeatedly follow

$$q \rightarrow \pi[q] \rightarrow \pi[\pi[q]] \rightarrow \dots,$$

you visit all valid smaller borders of P_q .

This explains why fallback using π is enough: no possible useful prefix is missed.

Border Chain

The prefix function creates a chain of fallback states:

$$q, \pi[q], \pi[\pi[q]], \dots, 0.$$

KMP follows this chain until it finds a state from which matching can continue.

13 Proof of the Prefix-Function Iteration Lemma

Proof. The value $\pi[q]$ is the largest proper border of P_q . Then $\pi[\pi[q]]$ is the largest proper border of that border, and so on. Therefore

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

are valid borders of P_q .

If j is any other proper border of P_q , then $j \leq \pi[q]$. If $j < \pi[q]$, then j is a border of $P_{\pi[q]}$, so the same argument continues on $\pi[q]$. Since every step strictly decreases the length, the chain lists all proper borders from largest to smallest. $\square$

Result

The fallback chain lists every reusable border:

$$q, \pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots, 0.$$

14 Correctness Idea of KMP

KMP can be viewed as simulating the finite-automaton matcher, but without explicitly storing the full transition table.

Instead of precomputing $\delta(q, a)$ for every state and alphabet character, KMP computes the needed transition on the fly using π .

Core Correctness Idea

After reading $T[1 : i]$, the variable q equals the length of the longest prefix of P that is also a suffix of $T[1 : i]$.

This is the same invariant used by the finite-automaton approach.

So when:

$$q = m,$$

the whole pattern has just matched.

15 Comparison with Previous Methods

Method	Preprocessing	Matching
Naive Matcher	0	$\mathcal{O}((n - m + 1)m)$
Rabin–Karp	$\Theta(m)$	average good, worst-case $\mathcal{O}((n - m + 1)m)$
Finite Automaton	$\mathcal{O}(m \Sigma)$ or more	$\Theta(n)$
KMP	$\Theta(m)$	$\Theta(n)$

Why KMP is Special

KMP keeps the linear-time matching performance of finite automata, but reduces preprocessing from $\mathcal{O}(m|\Sigma|)$ to:

$$\Theta(m).$$